FIAP ON

RAÍZES DA INTELIGÊNCIA - PREPARANDO O TERRENO

LINGUAGEM R E SUA ESPECIALIDADE

8

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Declaração de Variáveis 5
Código-fonte 2 - Sintaxe 5
Código-fonte 3 - Criação de uma nova variável 5
Código-fonte 4 - Atribuindo um valor a uma variável 6
Código-fonte 5 - Renomear uma variável em R 6
Código-fonte 6 - Manipulação de colunas 7
Código-fonte 7 - Atribuição de valores, renomear variáveis em uma data frame e criação de uma novas variáveis e seleção de colunas específicas em R 8
Código-fonte 8 - Valores ausentes 10
Código-fonte 9 - Remoção de linhas ou colunas com valores ausentes 10
Código-fonte 10 - Preenchimento de valores ausentes com médias, medianas ou outros valores relevantes 11
Código-fonte 11 - Utilização de técnicas avançadas de imputação 12
Código-fonte 12 - Exemplo do uso do dplyr para filtrar e resumir dados 13
Código-fonte 13 - Formato de dados 15
Código-fonte 14 - Valores ausentes 16
Código-fonte 15 - Operações de manipulação de dados 16

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 LINGUAGEM R E SUA ESPECIALIDADE 4
1.1 Criação de variáveis 5
1.2 Renomear variáveis 6
1.3 Manipulação de colunas 7
1.4 Aplicação 7
2 LIDANDO COM VALORES AUSENTES 9
2.1 Pacotes do R para manipulação de dados 12
2.2 Formato de dados largo 13
2.3 Formato de dados longo 14
2.4 Vantagens de cada formato 14
REFERÊNCIAS 17

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 LINGUAGEM R E SUA ESPECIALIDADE

Nas discussões de Data Science usamos a expressão “futuro eu” que, no contexto de programação, refere-se a “você mesmo” em um ponto futuro no tempo enquanto continua a desenvolver um projeto ou trabalhar em código. Em outras palavras, é a sua versão que lidará com o código ou projeto em algum momento posterior, após ter realizado mudanças, adicionado novos recursos, resolvido problemas ou feito outras modificações (Alby, 2023). A ideia por trás de se referir ao “futuro eu” é reconhecer que, ao escrever código, você está criando algo que pode precisar ser mantido, entendido e modificado no futuro. Portanto, é importante escrever o código de forma clara, documentada e organizada, para que o “futuro eu” (ou outros desenvolvedores que possam trabalhar no código) possa entender e continuar o trabalho com facilidade (Wickham, *et al*., 2023). Algumas práticas comuns que podem ajudar a tornar o “futuro eu” (e outros) mais eficaz ao lidar com o código:

- **Comentários claros:** adicione comentários relevantes ao código para explicar a lógica, a finalidade de funções e partes complexas do código.
- **Padrões de codificação:** siga padrões de codificação consistentes para facilitar a leitura e a manutenção do código.
- **Documentação:** crie documentação para o projeto ou biblioteca, descrevendo como usá-la e quaisquer requisitos.
- **Versionamento:** utilize sistemas de controle de versão, como Git, para rastrear e gerenciar as mudanças no código ao longo do tempo.
- **Testes automatizados:** escreva testes automatizados para garantir que as alterações futuras não quebrem funcionalidades existentes.
- **Refatoração:** regularmente, faça a refatoração do código para melhorar sua qualidade e eficiência.

Ao adotar essas práticas, você está ajudando a garantir que o “futuro eu” e outros desenvolvedores possam trabalhar de forma mais eficaz e sem problemas no projeto, economizando tempo e evitando frustrações ao lidar com o código no futuro.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.1 Criação de variáveis

A criação de novas variáveis desempenha um papel importante na manipulação de dados e envolve a criação de uma nova variável a partir de valores existentes, por meio da aplicação de operações ou funções. A sintaxe para criar uma nova variável é a seguinte:

```
nome_da_variavel <- valor  # ou
nome_da_variavel = valor
```

Código-fonte 1 - Declaração de Variáveis
Fonte: Elaborado pelo autor (2024)

Em que:

```
nome_da_variavel é o nome da variável.
<- ou = é o operador de atribuição.
valor é o valor que você deseja atribuir à variável.
```

Código-fonte 2 - Sintaxe
Fonte: Elaborado pelo autor (2024)

A seguir um exemplo de como criar uma nova variável "idade_em_anos" com base na variável "idade_meses":

```
     # data frame de exemplo com uma coluna "idade_meses"
     data_frame <- data.frame(idade_meses = c(12, 24, 36, 48,
60))

     # Mostrar o data frame original
     print("Data frame original:")
     print(data_frame)

     # Criar uma nova variável chamada "idade_em_anos" com base
em "idade_meses"
     data_frame$idade_em_anos <- data_frame$idade_meses / 12

     # Mostrar o data frame com a nova variável
     print("Data frame com a nova variável:")
     print(data_frame)
```

Código-fonte 3 - Criação de uma nova variável
Fonte: Elaborado pelo autor (2024)

Na linguagem R, a atribuição de valores a variáveis é simples e direta. É utilizado o operador <- ou o operador = para atribuir um valor a uma variável. Alguns exemplos seriam:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Atribuição de valores
idade <- 30
nome <- "João"
salario <- 50000.50
```

Código-fonte 4 - Atribuindo um valor a uma variável
Fonte: Elaborado pelo autor (2024)

1.2 Renomear variáveis

A estratégia de renomear variáveis é útil para tornar os nomes das variáveis mais descritivos ou limpar dados com nomes inadequados. Para renomear uma variável em R, a função colnames() pode ser utilizada para data frames ou simplesmente atribuir um novo nome à variável:

```
# Renomear uma variável em um data frame
colnames(data_frame)[colnames(data_frame)              ==
"variavel_antiga"] <- "variavel_nova"
# Ou, usando a função `names()`
names(data_frame)["variavel_antiga"] <- "variavel_nova"

# Exemplo
data_frame <- data.frame(
  nome = c("Alice", "Bob", "Charlie"),
  idade = c(30, 25, 35),
  cidade = c("Nova York", "Los Angeles", "Chicago")
)

# Mostrar o data frame original
print("Data frame original:")
print(data_frame)

# Renomear a variável "cidade" para "localidade" usando
colnames()
colnames(data_frame)[colnames(data_frame) == "cidade"] <-
"localidade"

# Mostrar o data frame com a variável renomeada
print("Data frame com a variável renomeada:")
print(data_frame)
```

Código-fonte 5 - Renomear uma variável em R
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.3 Manipulação de colunas

Pode ocorrer, em algumas fases dos projetos, a necessidade de trabalhar apenas com colunas específicas em um conjunto de dados. Será importante, então, fazer análises com critérios pré-estabelecidos da base de dados, o que pode ser feito com a seleção das colunas com base em seus nomes. Analise o script a seguir:

```
# data frame de exemplo
data_frame <- data.frame(
  nome = c("Alice", "Bob", "Charlie"),
  idade = c(30, 25, 35),
  cidade = c("Nova York", "Los Angeles", "Chicago")
)

# Mostrar o data frame original
print("Data frame original:")
print(data_frame)

# Selecionar uma única coluna
coluna_selecionada <- data_frame$idade

# Mostrar a coluna selecionada
print("Coluna selecionada:")
print(coluna_selecionada)

# Selecionar várias colunas
colunas_selecionadas <- data_frame[, c("nome", "cidade")]

# Mostrar as colunas selecionadas
print("Colunas selecionadas:")
print(colunas_selecionadas)
```

Código-fonte 6 - Manipulação de colunas
Fonte: Elaborado pelo autor (2024)

1.4 Aplicação

Para uma melhor compreensão da manipulação das variáveis, a sintaxe a seguir abrange os tópicos de atribuição de valores às variáveis, renomeação de variáveis, criação de novas variáveis e seleção de colunas específicas em R:

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     # Atribuição de valores a variáveis
     idade <- 2
     # Renomear variáveis em um data frame
     data_frame <- data.frame(var_antiga = c(1, 2, 3, 4, 5))
     colnames(data_frame)[colnames(data_frame)             ==
"var_antiga"] <- "var_nova"
     # Criação de uma nova variável com base em valores
existentes
     idade_em_meses <- idade * 12
```

Código-fonte 7 - Atribuição de valores, renomear variáveis em uma data frame e criação de uma novas variáveis e seleção de colunas específicas em R
Fonte: Elaborado pelo autor (2024)

Na sintaxe descrita, a lógica utilizada foi:

- O valor 25 foi atribuído à variável "idade".
- Foi renomeada uma coluna em um data frame de "var_antiga" para "var_nova".
- A nova variável "idade_em_meses" foi criada calculando a idade multiplicada por 12.
- As colunas "nome" e "idade" de um data frame foram selecionadas e armazenadas em "colunas_selecionadas".

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 LIDANDO COM VALORES AUSENTES

Valores ausentes em conjuntos de dados desempenham um papel crítico na manipulação e análise de dados, pois indicam informações ausentes em um conjunto de dados. Esses valores podem impactar significativamente a qualidade e a confiabilidade das análises e dos resultados. Assim, ignorar ou tratar inadequadamente valores ausentes pode resultar em conclusões incorretas ou enviesadas.

O primeiro passo é identificar os valores ausentes. Saber onde estão e quantos são é fundamental para tomar decisões sobre como abordar. O tratamento adequado envolve estratégias como preenchimento com valores válidos, remoção de linhas ou colunas afetadas ou imputação estatística. Além do tratamento de valores ausentes, a limpeza de dados inclui a detecção e correção de erros, a remoção de duplicatas e a garantia de que os dados estejam em um formato consistente. A importância da limpeza de dados se estende a todas as etapas de um projeto de análise de dados, desde a coleta até a apresentação dos resultados.

Dessa forma, a qualidade dos resultados em qualquer análise de dados está diretamente relacionada à qualidade dos dados subjacentes. Dados sujos, inconsistentes ou com valores ausentes podem introduzir vieses, erros e imprecisões em análises estatísticas e modelos de Machine Learning, que serão discutidos na sequência. A limpeza de dados capacita analistas e cientistas de dados a tomar decisões mais informadas e confiáveis (Fávero, Belfiore e Souza, 2023). Dados limpos e bem tratados são a base para a geração de insights precisos e conclusões válidas.

Na linguagem R, os valores ausentes são frequentemente representados como NA (Not Available) ou NULL. Para identificá-los em um data frame ou vetor é utilizada a função is.na():

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Identificação de valores ausentes em um vetor
vetor <- c(1, 2, NA, 4, 5)
valores_ausentes <- is.na(vetor)

# Identificação de valores ausentes em um data frame
data_frame <- data.frame(idade = c(25, NA, 30, 35),
                         salario  =  c(50000,  55000,  NA,
60000))
valores_ausentes <- is.na(data_frame)
print(valores_ausentes)
```

Código-fonte 8 - Valores ausentes
Fonte: Elaborado pelo autor (2024)

O tratamento de valores ausentes é essencial para garantir a qualidade e a precisão de suas análises. Existem várias estratégias para lidar com valores ausentes, como:

- **Remoção de linhas ou colunas com valores ausentes:** é uma abordagem válida quando os dados ausentes não podem ser imputados de forma significativa. Exemplo: considere um data frame chamado "dados" e o objetivo seria remover todas as linhas que contenham pelo menos um valor ausente:

```
# data frame de exemplo
dados <- data.frame(
  A = c(1, 2, NA, 4),
  B = c(NA, 2, 3, 4)
)

# Remover linhas com valores ausentes
dados_limpos <- na.omit(dados)

# Mostrar o data frame após a remoção
print(dados_limpos)
```

Código-fonte 9 - Remoção de linhas ou colunas com valores ausentes
Fonte: Elaborado pelo autor (2024)

- **Preenchimento de valores ausentes com médias, medianas ou outros valores relevantes:** usar estatísticas resumidas, como a média ou a mediana dos valores da coluna. A mediana é uma medida robusta que não é afetada por valores extremos (outliers) e pode ser uma escolha adequada para o preenchimento de valores ausentes quando a distribuição dos dados

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

é assimétrica. Exemplo: preencher valores ausentes na coluna "A" com a mediana dos valores existentes:

```
# data frame de exemplo
dados <- data.frame(
  A = c(1, 2, NA, 4, 5),
  B = c(NA, 2, 3, 4, 5)
)

# Calcular a mediana da coluna "A"
mediana <- median(dados$A, na.rm = TRUE)

# Preencher valores ausentes com a mediana
dados$A[is.na(dados$A)] <- mediana

# Mostrar o data frame após o preenchimento
print(dados)
```

Código-fonte 10 - Preenchimento de valores ausentes com médias, medianas ou outros valores relevantes
Fonte: Elaborado pelo autor (2024)

- **Utilização de técnicas avançadas de imputação:** técnicas avançadas, como imputação com modelos de Machine Learning, podem ser utilizadas para preencher valores ausentes de forma mais sofisticada. Exemplo: usar o pacote mice para imputação de valores ausentes utilizando a regressão linear:

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     # Instalar e carregar o pacote mice
     install.packages("mice")
     library(mice)

     # Carregar o data frame de dados
     # O dataframe nhanes é um dataframe de dados de saúde que
contém valores ausentes
     data(nhanes)

     # Visualizar o data frame de dados
     head(nhanes)

     # Definir o método de imputação
     # O método pmm é um método de imputação por regressão
linear que usa uma média ponderada das variáveis não ausentes
para estimar os valores ausentes.
     impute_method <- "pmm"

     # Realizar a imputação
     # Este passo realiza a imputação de valores ausentes
usando
     # o método especificado em impute_method.
     nhanes_imputed <- mice(nhanes, method = impute_method)

     # Visualizar os dados imputados
     completed_data <- complete(nhanes_imputed)
     head(completed_data)
```

Código-fonte 11 - Utilização de técnicas avançadas de imputação
Fonte: Elaborado pelo autor (2024)

2.1 Pacotes do R para manipulação de dados

Os dois pacotes mais populares para a manipulação de dados são **dplyr** e **tidyr**. Essas bibliotecas são utilizadas por cientistas de dados para simplificar tarefas de manipulação de dados e transformar conjuntos de dados de maneira eficiente. O dplyr é uma biblioteca R que fornece um conjunto de funções simples e consistentes para realizar operações de manipulação de dados em data frames (Jockers, *et al*., 2020). Entre as principais funções do dplyr estão:

- **filter():** filtra linhas com base em condições específicas.
- **select():** seleciona colunas específicas.
- **mutate():** cria novas colunas com base em operações em colunas existentes.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **group_by():** agrupa os dados por uma ou mais variáveis.
- **summarize():** calcula estatísticas resumidas em grupos de dados.

Exemplo de uso do dplyr para filtrar e resumir dados:

```
# Carregar a biblioteca dplyr
library(dplyr)

# Criar dados
# mulheres - 0
# homens - 1
data_frame <- data.frame(
  idade = c(30, 25, 35, 50, 15, 20),
  genero = c(0, 0, 1, 1, 0, 1)
)

# Filtrar dados: selecionar apenas linhas onde a idade
seja maior que 25
dados_filtrados <- data_frame %>% filter(idade > 25)

# Agrupar dados por gênero e calcular a média de salário
por grupo
dados_agrupados <- data_frame %>%
  group_by(genero) %>%
  summarize(media_idade = mean(idade))

# Visualizar resultado
Print(dados_agrupados)
```

Código-fonte 12 - Exemplo do uso do dplyr para filtrar e resumir dados
Fonte: Elaborado pelo autor (2024)

O tidyr é uma biblioteca R que se concentra na organização de dados, principalmente na transformação entre formatos de dados "largos" e "longos".

2.2 Formato de dados largo

Um formato de dados largo é aquele em que cada coluna representa uma variável. Por exemplo, um data frame com as colunas id, data, valor1 e valor2 está no formato largo. Cada coluna representa uma variável diferente: id representa o identificador do registro, data representa a data do registro, valor1 representa o valor da variável 1 e valor2 representa o valor da variável 2.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.3 Formato de dados longo

Um formato de dados longo é aquele em que cada linha representa um registro. Por exemplo, um data frame com as colunas id, variável, valor está no formato longo. Cada linha representa um registro diferente, e a coluna variável representa o nome da variável cujo valor está na coluna valor.

2.4 Vantagens de cada formato

Cada formato de dados possui suas vantagens e desvantagens. O formato largo é geralmente mais compactado, pois as colunas são independentes. O formato longo é geralmente mais flexível, pois permite a análise de dados multivariados.

Neste sentido, o tidyr fornece as funções gather() e spread() para transformar dados de um formato para outro. A função gather() transforma dados de formato largo para formato longo, e a função spread() transforma dados de formato longo para formato largo.

```
     # Instalar e carregar o pacote dplyr e tidyr
     install.packages("tidyr", "dplyr")
     library(tidyr)
     library(dplyr)

     # Vamos usar os comandos gather() e spread() do pacote
tidyr para converter
     # um data frame de formato "largo" para "longo" e vice-
versa.

     # dataframe largo
     # id: Identificador do registro
     # data: Data do registro
     # valor1: Valor da variável 1
     # valor2: Valor da variável 2

     # Criar um data frame de formato "largo"
     dados <- data.frame(
       id = c(1, 2, 3, 4, 5),
       data  =  c("2023-07-20",  "2023-07-21",  "2023-07-22",
"2023-07-23", "2023-07-24"),
       valor1 = c(10, 20, 30, 40, 50),
       valor2 = c(20, 40, 60, 80, 100)
     )
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     # Convertendo de formato "largo" para "longo"
     # Para converter o data frame de formato "largo" para
"longo", podemos usar o
     # comando gather(). O comando gather() usa as seguintes
especificações:

     # key: Nome da coluna que conterá os nomes das variáveis
     # value: Nome da coluna que conterá os valores das
variáveis
     # id: Colunas que não devem ser incluídas no formato
"longo"

     # Convertendo de formato "largo" para "longo"
     dados_longos <- dados %>%
       gather(key = variavel, value = valor, -id, -data)

     # Visualizar as primeiras linhas do dataframe
"dados_longos"
     head(dados_longos)

     # Convertendo de formato "longo" para "largo"
     # Para converter o data frame de formato "longo" para
"largo", podemos usar
     # o comando spread(). O comando spread() usa as seguintes
especificações:
     # key: Nome da coluna que contém os nomes das variáveis
     # value: Nome da coluna que contém os valores das
variáveis

     dados_largos <- dados_longos %>%
       spread(key = variavel, value = valor)

     # Visualizar as primeiras linhas do dataframe
"dados_largos"
     head(dados_largos)
```

Código-fonte 13 - Formato de dados
Fonte: Elaborado pelo autor (2024)

Os pacotes não apenas simplificam as tarefas de manipulação de dados, mas também tornam o código mais claro e eficiente. Dominar as bibliotecas dplyr e tidyr permitirá que você realize operações de manipulação de dados com facilidade no ambiente R.

Para ilustrar como abordar valores ausentes e usar esses pacotes de manipulação de dados, vamos considerar um conjunto de dados que contém informações sobre vendas em uma loja, mas que apresenta valores ausentes que precisam ser tratados. Em seguida, demonstraremos como o pacote dplyr pode ser

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

utilizado para realizar manipulações nos dados. Suponha um conjunto de dados chamado "vendas_loja" com as seguintes colunas: "data_venda", "produto", "quantidade" e "receita". No entanto, este conjunto de dados possui valores ausentes. O primeiro passo será lidar com os valores ausentes.

```
# Carregue a biblioteca dplyr
library(dplyr)

# Crie um data frame com valores ausentes
vendas_loja <- data.frame(
  data_venda = c("2023-01-01", "2023-01-02", "2023-01-
03", "2023-01-04", "2023-01-05"),
  produto = c("A", "B", "A", NA, "C"),
  quantidade = c(10, 15, NA, 20, 25),
  receita = c(100, 150, 200, NA, 250)
)

# Identifique e conte os valores ausentes em cada coluna
valores_ausentes <- colSums(is.na(vendas_loja))
print(valores_ausentes)
```

Código-fonte 14 - Valores ausentes
Fonte: Elaborado pelo autor (2024)

No próximo passo, o pacote dplyr será utilizado para realizar operações de manipulação de dados no conjunto de dados "vendas_loja".

```
# 1. Remova as linhas com valores ausentes
vendas_limpas <- vendas_loja %>%
  drop_na()

# 2. Calcule a média da quantidade vendida
media_quantidade <- vendas_loja %>%
  summarize(media_quantidade = mean(quantidade, na.rm =
TRUE))

# 3. Crie uma nova coluna chamada "valor_total"
multiplicando a quantidade pela receita
vendas_com_valor_total <- vendas_loja %>%
  mutate(valor_total = quantidade * receita)

# 4. Selecione apenas as colunas "data_venda" e "produto"
colunas_selecionadas <- vendas_loja %>%
  select(data_venda, produto)
```

Código-fonte 15 - Operações de manipulação de dados
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

ALBY, T. **Data Science in Practice**. [*s.l.*] CRC Press, 2023.

JOCKERS, M. L.; THALKEN, R. **Text Analysis with R. : For Students of Literature**. Cham: Springer International Publishing, 2020.

LUIZ PAULO FAVERO; BELFIORE, P.; RAFAEL. **Data Science, Analytics and Machine Learning with R**. [*s.l.*] Academic Press, 2023.

WICKHAM, H.; MINE ÇETINKAYA-RUNDEL; GROLEMUND, G. **R for Data Science**. [*s.l.*] O'Reilly Media, 2023.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com